

Laundry Marketplace Platform

A two-sided marketplace connecting Clients with independent LaundryMart vendors for on-demand wash, fold, and delivery service.

Prepared for: Engineering, Design, QA, DevOps & Product · Status: Ready for implementation

1. Executive Summary

This document specifies a production-ready, two-sided marketplace web application built on the MERN stack (MongoDB, Express, React, Node.js). The platform has exactly two user roles for MVP — **Client** (customer) and **LaundryMart** (vendor) — with no administrative panel. Clients discover nearby LaundryMarts using geolocation search, compare services and pricing, schedule pickup and delivery, pay securely online, and track orders in real time. LaundryMarts manage their business profile, service catalog, pricing, and incoming orders through a dedicated dashboard.

The document below is organized as the single source of truth for the project: it covers product scope, every frontend screen with its states and validation rules, the complete backend architecture and database schema, the full REST API surface, the reusable UI component library, core user flows, security and testing strategy, and a phased delivery roadmap. Each functional requirement includes business logic, validation rules, acceptance criteria, and edge cases so that no team needs additional clarification to begin building.

2. Product Vision & Scope

2.1 Vision

Make professional laundry service as easy to book as a rideshare: find a trusted local LaundryMart, schedule a pickup in under a minute, and track the order until clean laundry is back at the door — with transparent, itemized pricing on both sides of the marketplace.

2.2 In Scope (MVP)

- Client and LaundryMart authentication, each with its own dashboard
- Geolocation-based vendor search, filtering, and sorting
- Vendor service catalogs with per-item pricing set by the vendor
- Booking, scheduled pickup/delivery windows, and checkout with online payment

- Order lifecycle management and live status tracking for both roles
- Ratings & reviews, notifications, and profile/business settings

2.3 Out of Scope (MVP)

- Admin panel / internal ops console (flagged for Phase 2 — see §20)
- Native mobile apps (responsive web only for MVP)
- In-app driver/logistics fleet management (pickup/delivery is arranged by the LaundryMart itself in MVP)
- Multi-currency support, subscriptions/recurring billing, loyalty programs

3. Technology Stack

Layer	Choice	Notes
Frontend	React.js (JS, ES6+), React Router, Context API	Redux is not needed at MVP scale; revisit only if cross-cutting state (e.g. real-time order status across many components) outgrows Context.
Styling	Tailwind CSS	Utility classes; shared design tokens in <code>tailwind.config.js</code> .
HTTP client	Axios	Centralized instance with interceptors for JWT attach + 401 refresh/redirect.
Backend	Node.js, Express.js	REST architecture, layered controller/service pattern.
Database	MongoDB + Mongoose ODM	2dsphere geospatial indexes for vendor search.
Auth	JWT (access + refresh), bcrypt.js	Access token 15 min, refresh token 30 days, httpOnly cookie.
File uploads	Multer + Cloudinary	Multer buffers to memory, streams to Cloudinary; no local disk writes.
Validation	express-validator	Schema-level validators run before controller logic on every mutating route.
Payments	Stripe	See §4 for full rationale and workflow.

4. Payment Gateway: Stripe vs. Flutterwave

Recommendation: Stripe.

Criteria	Stripe	Flutterwave
Marketplace payouts	Stripe Connect natively splits a payment between platform fee and vendor payout, with vendor onboarding (KYC) built in.	Sub-account/split payment support exists but is less mature for automated marketplace payouts.
Docs & SDKs	Best-in-class Node.js SDK, Elements for PCI-compliant card capture, extensive test tooling.	Solid SDK, smaller ecosystem, fewer pre-built UI primitives.
Geographic fit	Strongest in US/EU/UK markets.	Strongest in Africa; not the target launch geography here.
Compliance	PCI DSS SAQ-A eligible via Elements/Payment Element (card data never touches our server).	Comparable, but fewer client libraries reach the same SAQ-A simplicity.

Given the target markets (US launch cities), Stripe is the better fit — primarily because of Stripe Connect, which is purpose-built for exactly this app's shape: a platform that collects payment from a Client and pays out a percentage to a LaundryMart vendor.

4.1 Integration Workflow

- 1. Vendor onboarding:** On business registration, create a Stripe Connect Express account for the LaundryMart (`stripe.accounts.create({'type': 'express'})`); redirect to Stripe-hosted onboarding link to collect KYC/bank details. Store `stripeAccountId` on the Vendor document.
- 2. Checkout:** Client submits card via Stripe Elements (client-side tokenization; raw card data never hits our API). Frontend sends the resulting `paymentMethodId` to `POST /api/payments/create-intent`.
- 3. PaymentIntent with destination charge:** Backend creates a `PaymentIntent` with `application_fee_amount` (platform commission, default 15%) and `transfer_data.destination` set to the vendor's connected account ID — Stripe splits funds automatically at settlement.
- 4. Confirmation:** Client confirms via `stripe.confirmCardPayment` ; on success the Order status moves from `pending_payment` → `confirmed`.
- 5. Webhooks (source of truth):** Listen for `payment_intent.succeeded` , `payment_intent.payment_failed` , and `account.updated` at `POST /api/webhooks/stripe` ; verify signature with the webhook secret before trusting the payload. Never mark an order paid from the client-side confirmation alone.
- 6. Refunds:** Triggered from an order cancellation within the cancellation window (\$9); calls `stripe.refunds.create` with `reverse_transfer: true` to claw back the vendor's portion too.

5. User Roles & Authentication

Exactly two roles for MVP: **Client** and **LaundryMart**. No admin role. Role is stored on the User document (`role: 'client' | 'vendor'`) and encoded in the JWT payload; middleware checks it on every protected route (RBAC, §14).

5.1 Client Authentication Flow

1. **Sign Up:** name, email, phone, password, confirm password. Validation: email format + uniqueness check (async, on blur); password ≥ 8 chars with 1 number + 1 letter; phone via libphonenumber pattern. On submit: `POST /api/auth/client/register` hashes password (bcrypt, 12 rounds), creates User, sends verification email, returns access+refresh tokens. Edge case: duplicate email returns 409 with a field-level error, not a generic toast.
2. **Login:** email + password $\rightarrow$ `POST /api/auth/client/login` . Rate-limited to 5 attempts / 15 min per IP+email pair; on the 6th, respond 429 with retry-after.
3. **Forgot Password:** email $\rightarrow$ `POST /api/auth/forgot-password` issues a signed, single-use reset token (JWT, 1-hour expiry) emailed as a link. Always returns 200 (never reveal whether the email exists).
4. **Reset Password:** token + new password $\rightarrow$ `POST /api/auth/reset-password` verifies token signature + expiry + single-use flag, updates password hash, invalidates all existing refresh tokens for that user.
5. **Logout:** `POST /api/auth/logout` clears the httpOnly refresh cookie and blacklists the refresh token server-side.

5.2 LaundryMart Authentication Flow

1. **Register Business:** multi-step form — (1) owner name, email, phone, password; (2) business name, address (geocoded to lat/lng via Google Geocoding API), business hours; (3) Stripe Connect onboarding redirect. Vendor record is created with `status: 'pending_verification'` until Stripe onboarding completes (webhook flips it to `active`) — a pending vendor is not shown in client search results.
2. **Login / Forgot / Reset / Logout:** identical mechanics to Client flows above, on vendor-scoped routes (`/api/auth/vendor/*`) so tokens are minted with `role: 'vendor'` .

Acceptance criteria (both roles): a successful auth call returns a user/vendor object (no password hash), sets the refresh token as httpOnly+Secure+SameSite=Lax, and the access token in the response body for the client to hold in memory (never localStorage, to reduce XSS token theft surface).

6. Frontend Pages — Public

Page	Purpose & Key Components	States & Behavior
Landing	Navbar, Hero with location-aware "Find LaundryMarts near you" search bar, How-it-works, featured vendor cards, testimonials, CTA banner, Footer.	Loading: skeleton for featured vendors. Empty: generic "Coming to your area soon" if geolocation resolves outside launch cities. Responsive: hero stacks vertically <768px.
About	Static brand/mission content, team or founding story section.	No dynamic states — static content page.
Services	Explains service categories (wash & fold, dry cleaning, bedding) generically; CTA to search vendors.	Static; CTA routes to search with geolocation prompt.
Contact	Contact form (name, email, subject, message) + support email/phone.	Loading: submit button spinner. Success: inline confirmation, form clears. Error: inline banner, form retains input.
FAQs	Accordion grouped by topic (Clients, Vendors, Payments).	Client-side only; one panel open at a time on mobile.
Privacy Policy / Terms	Long-form legal text, table of contents anchor nav.	Static; print-friendly styling.

7. Frontend Pages — Authentication

Page	Validation Rules	States
Client Sign Up	Required fields; email format + async uniqueness; password strength meter; ToS checkbox required.	Inline field errors on blur; submit disabled until valid; loading spinner on submit; success redirects to Client Dashboard.
Client Login	Required email + password; generic "invalid credentials" (never reveal which field is wrong).	Error banner on 401; lockout notice on 429 with countdown.
Vendor Registration	3-step wizard with step-level validation; address must resolve via geocoding before step 2 can advance.	Progress indicator; step state persisted in sessionStorage so a refresh doesn't lose progress; final step shows Stripe onboarding redirect loading state.
Vendor Login	Same as Client Login, vendor-scoped endpoint.	If Stripe onboarding incomplete, redirect to a "finish setup" screen instead of the dashboard.
Forgot / Reset Password	Email format (forgot); password strength + match (reset); reset token must be present in URL.	Forgot always shows the same "check your email" success state. Reset shows an expired/invalid-token error state with a re-request link.

8. Frontend Pages — Client Dashboard

Page	Purpose & Components	States / Logic
Dashboard Overview	Active order status card, quick-rebook shortcuts, recent activity feed.	Empty: "No orders yet" with a search CTA for first-time users.
Nearby LaundryMarts	Map + list toggle, vendor cards (rating, distance, price range, turnaround time).	Requires geolocation permission; fallback to manual ZIP entry if denied. Empty: no vendors within radius → suggest widening radius.
Search Vendors	Filters (distance, price, rating, service type, turnaround), sort (distance/rating/price).	Skeleton cards while loading; filters persist in URL query params for shareable/back-button-safe state.
Vendor Details	Business profile, photo gallery, service list with prices, reviews, hours, "Book Now" CTA.	Closed-now banner if outside business hours (booking still allowed for next window). Error: vendor not found → 404 state with back-to-search link.
Book Laundry Service	Service picker with per-item quantity/weight estimator, running subtotal.	Validation: at least 1 service line item required to proceed; quantity must be > 0.
Schedule Pickup	Date & time-window picker (pickup + delivery), address selector/entry, special instructions field.	Validation: only future slots within vendor's published availability are selectable; delivery window must be ≥ vendor's minimum turnaround after pickup.
Checkout	Order summary, promo code field, tip selector, total breakdown (subtotal, fee, tip, tax).	Recalculates live as tip/promo change; "Place Order" disabled until payment step validates.
Payment	Stripe Payment Element, saved cards list, add-new-card flow.	Loading: processing overlay blocks double-submit. Error: card-decline message inline from Stripe's error object, order remains editable.
Order Tracking	Status timeline (Placed → Accepted → Picked Up → In Progress → Ready → Delivered), live map placeholder, vendor contact button.	Polls/subscribes for status changes; each transition pushes a Notification (see §12).
Order History	Paginated table/list of past orders, filter by status/date, "Reorder" and "Leave a review" actions.	Empty state for first-time users; pagination via cursor, 20/page.
Notifications	Chronological list, unread badge, mark-all-read.	Empty: "You're all caught up" illustration state.
Profile	Name, email, phone, saved addresses, profile photo upload.	Inline field-level save (autosave on blur) with toast confirmation.

Page	Purpose & Components	States / Logic
Settings	Notification preferences, password change, saved payment methods, delete-account flow.	Delete-account requires typed confirmation + password re-entry.

9. Frontend Pages — LaundryMart Dashboard

Page	Purpose & Components	States / Logic
Dashboard Overview	Today's orders count, pending requests badge, earnings snapshot, rating summary.	Onboarding checklist shown until profile + first service + Stripe setup are all complete.
Manage Orders	Kanban-style columns (New, Accepted, In Progress, Ready, Completed) or filterable table; per-order action buttons.	Empty per column. New orders bordered/highlighted with a response-time indicator (auto-cancel if not actioned within X minutes, per §9 business rules below).
Manage Services	CRUD list of services (name, category, unit — per lb/per item, price, est. turnaround), add/edit modal.	Validation: price > 0, name required, at least one active service required before going live.
Update Prices	Bulk price-editing table across all services with percentage adjust helper.	Confirms before applying bulk change; changes logged with timestamp for dispute resolution.
Manage Availability	Weekly hours grid, blackout-date picker, max daily order capacity.	Closing early/blackout dates immediately removes those slots from Client scheduling.
Earnings	Revenue chart (daily/weekly/monthly), payout history, platform fee breakdown, CSV export.	Pulls from Stripe Connect balance/payout API; empty state before first completed order.
Customer Requests	Inbound messages/special requests tied to an order thread.	Unread badge; read-receipt on open.
Notifications	New order, cancellation, review, and payout notifications.	Same pattern as Client Notifications (§8).
Business Profile	Business name, description, logo + gallery upload (Cloudinary), address, contact info.	Image upload: max 5MB/file, jpg/png/webp only, client-side compression before upload.
Settings	Password change, payout bank details (deep-links to Stripe Express dashboard), notification prefs, deactivate business.	Deactivating hides the vendor from search but preserves historical order data.

10. Functional Requirements & Business Rules

10.1 Order Lifecycle (Business Logic)

Order status enum: `pending_payment` → `confirmed` → `accepted` → `picked_up` → `in_progress` → `ready_for_delivery` → `delivered` → `completed` , with terminal branches `cancelled` and `disputed` .

- **Acceptance window:** a vendor has 15 minutes to accept or reject a new order; if it times out, the order auto-cancels and the Client is refunded in full and prompted to rebook with another vendor.
- **Cancellation policy:** Client may cancel free of charge up until the vendor marks `picked_up` ; after that, a cancellation requires vendor approval and may incur a partial charge for work already done.
- **Reviews:** only enabled on orders in `completed` status, one review per order, editable within 48 hours of submission.
- **Platform fee:** default 15% commission on order subtotal (configurable per-vendor field for future negotiated rates), computed server-side — never trust a client-submitted total.
- **Geosearch radius:** default 8 miles, user-adjustable up to 25 miles; vendors beyond that are excluded from results regardless of rating.

10.2 Client Features

- Register/login, geolocation vendor search with filter/sort, vendor profile comparison
- Schedule pickup/delivery, place orders, pay securely, track live status, view order history
- Rate & review completed orders, manage notifications and profile

10.3 LaundryMart Features

- Register business, manage profile + logo/gallery uploads, manage services and pricing
- Accept/reject orders, update order status through the lifecycle, view earnings and payouts
- Respond to customer requests, manage notifications

11. Backend Architecture

11.1 System Overview

A layered Express application: **routes** declare endpoints and attach validators/middleware → **controllers** parse the request and call **services** (business logic, framework-agnostic) → services talk to **Mongoose models**. Cross-cutting concerns (auth, error handling, logging, rate limiting) live in middleware applied globally or per-router.

11.2 Backend Folder Structure

```
server/  
├─ config/           # db connection, cloudinary, stripe, env loader  
├─ controllers/      # auth, client, vendor, order, payment, review, notification  
├─ models/           # User, Vendor, Service, Order, Payment, Review, Notification  
├─ routes/           # one router per resource, mounted under /api  
├─ middleware/       # auth (JWT verify + RBAC), validate, rateLimiter, errorHandler, upload  
                    (multer)  
├─ services/         # geocode.service, stripe.service, cloudinary.service,  
notification.service  
├─ validations/      # express-validator chains per route  
├─ utils/            # asyncHandler, ApiError, ApiResponse, logger  
├─ uploads/          # multer memory storage tmp (not persisted; streamed to Cloudinary)  
└─ server.js
```

11.3 Frontend Folder Structure

```
client/src/  
├─ components/       # shared UI (Button, Input, Modal, Toast, VendorCard...)  
├─ pages/            # one folder per route, e.g. pages/client/OrderTracking  
├─ layouts/          # ClientDashboardLayout, VendorDashboardLayout, PublicLayout  
├─ hooks/            # useAuth, useGeolocation, useDebounce, useOrderStatus  
├─ context/          # AuthContext, NotificationContext, CartContext  
├─ services/         # api/ axios wrappers per resource (authApi, orderApi...)  
├─ routes/           # AppRouter.jsx, ProtectedRoute.jsx, RoleRoute.jsx  
├─ utils/            # formatCurrency, validators, dateHelpers  
└─ assets/
```

12. Database Design

Collections use referenced relationships (ObjectId) rather than deep embedding, except where sub-documents are always accessed with their parent (e.g. OrderItems inside Order).

Users (Clients)

Field	Type	Notes
name	String	required, 2-60 chars
email	String	required, unique, lowercase, indexed
phone	String	required, E.164 format
passwordHash	String	required, select:false
role	String enum	'client' (default)
addresses	[[{"label,line1,city,state,zip,lat,lng"}]]	embedded array, max 10
stripeCustomerId	String	set on first checkout
avatarUrl	String	Cloudinary URL
isVerified	Boolean	default false
timestamps	Date	createdAt/updatedAt

Vendors (+ embedded Vendor Profile)

Field	Type	Notes
ownerName, email, phone, passwordHash	String	same rules as User
businessName	String	required, 2-80 chars
description	String	max 1000 chars
location	GeoJSON Point	<code>{ "type": "Point", "coordinates": [lng,lat] }</code> , 2dsphere indexed
address	<code>{ "line1", "city", "state", "zip" }</code>	required
logoUrl, galleryUrls	String, [String]	Cloudinary, gallery max 10 images
hours	<code>[{ "day", "open", "close" }]</code>	7 entries, closed = null
blackoutDates	[Date]	optional
commissionRate	Number	default 0.15
stripeAccountId, stripeOnboardingComplete	String, Boolean	gates <code>status: 'active'</code>
status	String enum	pending_verification / active / deactivated
ratingAvg, ratingCount	Number	denormalized, recomputed on new Review

Services

Field	Type	Notes
vendorId	ObjectId → Vendor	indexed
name, category	String	category enum: wash_fold / dry_clean / bedding / ironing / other
unit	String enum	per_lb / per_item
price	Number	required, min 0, stored in cents
estTurnaroundHrs	Number	used to compute earliest delivery slot
isActive	Boolean	soft-disable without deleting

Orders (+ embedded OrderItems)

Field	Type	Notes
clientId, vendorId	ObjectId	both indexed
items	[[{"serviceId,name,unit,unitPrice,qty,lineTotal"}]]	snapshot at time of order — price changes later don't retroactively affect placed orders
pickupWindow, deliveryWindow	{start,end} Date	required
address	embedded (copy of chosen address)	snapshot, not a live reference
subtotal, platformFee, tip, tax, total	Number (cents)	all server-computed, never trusted from client
status	String enum	see §10.1 lifecycle
statusHistory	[[{"status,at"}]]	audit trail, powers the tracking timeline
paymentId	ObjectId → Payment	1:1

Payments

Field	Type	Notes
orderId	ObjectId → Order	indexed, unique
stripePaymentIntentId	String	unique, source of truth for status
amount, applicationFee	Number (cents)	mirror of Stripe object for reporting
status	String enum	requires_payment_method / succeeded / failed / refunded
refundedAmount	Number	default 0, supports partial refunds

Reviews

Field	Type	Notes
orderId, clientId, vendorId	ObjectId	orderId unique (1 review/order)
rating	Number	required, 1-5 integer
comment	String	optional, max 500 chars
editableUntil	Date	createdAt + 48h

Notifications

Field	Type	Notes
recipientId, recipientRole	ObjectId, String	polymorphic recipient
type	String enum	order_update / new_order / new_review / payout / message
title, body, linkTo	String	linkTo is a client-side route
isRead	Boolean	default false, indexed with recipientId for unread counts

13. API Documentation

13.1 Auth

Method & Route	Auth	Description	Responses
POST /api/auth/client/register	Public	Create client account	201 user+tokens · 409 email taken · 422 validation
POST /api/auth/client/login	Public	Client login	200 user+tokens · 401 invalid · 429 rate-limited
POST /api/auth/vendor/register	Public	Create vendor + business profile	201 vendor+tokens+stripeOnboardingUrl · 409 · 422
POST /api/auth/vendor/login	Public	Vendor login	200 · 401 · 429
POST /api/auth/refresh	Refresh cookie	Rotate access token	200 new access token · 401 expired/invalid
POST /api/auth/forgot-password	Public	Email reset link	200 always
POST /api/auth/reset-password	Reset token	Set new password	200 · 400 expired/invalid token
POST /api/auth/logout	Authenticated	Clear session	200

13.2 Vendors & Search

Method & Route	Auth	Description	Responses
GET /api/vendors? lat&lng&radius&sort&category&minRating	Public	Geo search, <code>\$geoNear</code> aggregation on 2dsphere index	200 paginated list · 422 bad coords
GET /api/vendors/:id	Public	Vendor detail incl. services + rating	200 · 404
PATCH /api/vendors/me	Vendor	Update business profile	200 · 422
POST /api/vendors/me/media	Vendor	Upload logo/gallery (multipart, Multer→Cloudinary)	200 urls · 413 too large · 415 bad type
PATCH /api/vendors/me/availability	Vendor	Update hours/blackout dates	200 · 422

13.3 Services, Orders, Payments, Reviews

Method & Route	Auth	Description	Responses
POST /api/services	Vendor	Create service	201 · 422
PATCH /api/services/:id · DELETE /api/services/:id	Vendor (owner only)	Edit/soft-delete service	200 · 403 not owner · 404
POST /api/orders	Client	Create order in <code>pending_payment</code> , server recomputes totals	201 · 422 invalid slot/items
POST /api/payments/create-intent	Client	Create Stripe PaymentIntent for order	200 clientSecret · 402 card error
POST /api/webhooks/stripe	Stripe signature	Confirm payment, flip order → confirmed	200 · 400 bad signature
GET /api/orders/me	Client or Vendor	Own orders, paginated + filterable	200
PATCH /api/orders/:id/status	Vendor (owner)	Advance status; validates legal transition only	200 · 409 illegal transition · 403
POST /api/orders/:id/cancel	Client or Vendor	Cancel + trigger refund per policy (§10.1)	200 · 409 past cutoff
POST /api/reviews	Client	Review a completed order	201 · 409 order not completed / already reviewed
GET /api/notifications · PATCH /api/notifications/:id/read	Authenticated	List / mark read	200

14. UI Component Library

Component	Notes
Navbar	Role-aware — shows auth links (guest) or dashboard/avatar menu (logged in); mobile hamburger drawer.
Hero Section	Headline + embedded Search Bar, illustrative background.
Search Bar	Location autocomplete (Google Places) + "use my location" button.
Vendor Card	Logo, name, rating stars, distance, price-range indicator, "open now" badge.
Service Card	Name, unit price, quantity stepper.
Booking Form	Composed of Service Card list + Date & Time Picker + Address selector.
Button	Primary / secondary / destructive / ghost variants; loading spinner state.
Input / Select / Dropdown	Shared label + error-message slot; consistent focus ring.
Date & Time Picker	Disables past dates and slots outside vendor availability.
Payment Modal	Wraps Stripe Payment Element; handles 3DS challenge iframe.
Order Status Timeline	Horizontal (desktop) / vertical (mobile) step indicator driven by statusHistory.
Table	Sortable headers, sticky header row, row-level actions menu.
Modal	Focus-trapped, ESC to close, backdrop click configurable.
Toast Notification	Success/error/info variants, auto-dismiss 4s, stackable.
Skeleton Loader	Card/list/table variants matching final content's shape.
Empty / Error States	Shared illustration placeholder + message + optional retry/CTA button.
Pagination	Cursor-based, prev/next + page-size selector.
Footer	Sitemap links, social icons, legal links.

15. Core User Flows

15.1 Client: Search → Book → Pay → Track

Landing → grant/deny geolocation → Nearby LaundryMarts (list+map) → apply filters → open Vendor Details → Book Laundry Service (select items) → Schedule Pickup (date/time/address) → Checkout (review totals, add tip/promo) → Payment (Stripe Element, 3DS if required) → Order confirmed → Order Tracking (live timeline) → Delivered → prompted to Leave Review.

15.2 Vendor: Registration → First Order

Register Business (3-step) → Stripe Connect onboarding (external redirect + return) → status flips to active on webhook → add ≥ 1 Service → set Availability → appears in Client search → receives New Order notification → Accept within 15-min window → update status through pickup/in-progress/ready/delivered → Earnings updates on webhook-confirmed payout.

15.3 Order Cancellation

Client or Vendor initiates cancel → system checks current status against cutoff (§10.1) → if before pickup: auto full refund via Stripe (`reverse_transfer:true`), order → cancelled, both parties notified → if after pickup: requires Vendor approval of a partial refund amount before the refund fires.

16. Security Requirements

- **JWT:** short-lived access token (15 min) in memory; refresh token (30 days) as `httpOnly+Secure+SameSite=Lax` cookie; refresh rotation with reuse detection (a reused refresh token revokes the whole family).
- **Password hashing:** bcrypt, 12 salt rounds; never log or return password hashes (`select:false` on schema).
- **RBAC:** middleware reads `req.user.role` from verified JWT; route-level `requireRole('vendor')` guards; resource-level ownership checks (e.g. a vendor can only PATCH their own Service/Order).
- **Input validation:** express-validator chains on every mutating route; reject unknown fields; sanitize/escape free-text fields (review comments, order instructions) before storage.
- **File uploads:** Multer memory storage, 5MB limit, MIME allowlist (jpg/png/webp), re-encode via Cloudinary (strips EXIF/GPS metadata automatically).
- **Rate limiting:** global 100 req/min/IP; stricter 5 req/15min on auth endpoints; keyed by IP+identifier to prevent credential-stuffing.
- **NoSQL injection:** Mongoose schema typing rejects operator-shaped input by default; additionally strip `$ / .`-prefixed keys from all request bodies via sanitization middleware.
- **XSS:** React's default escaping + Content-Security-Policy header; never `dangerouslySetInnerHTML` on user-supplied text.
- **CSRF:** SameSite=Lax cookie + custom header requirement on state-changing requests mitigates CSRF without a separate token scheme.
- **CORS:** allowlist the production frontend origin only; `credentials:true` for cookie-based refresh.
- **Secrets:** all keys (Stripe, Cloudinary, JWT secret, Mongo URI) in environment variables, never committed; separate test/live Stripe keys per environment.

17. Testing Strategy

Level	Tooling	Coverage focus
Unit	Jest	Services (price calc, order-status transitions, geocode helpers), React components (RTL)
Integration	Jest + Supertest + mongodb-memory-server	Route → controller → DB round trips, RBAC enforcement
API	Postman/Newman collection in CI	Contract tests against every documented endpoint in §13
E2E	Playwright	Full flows: client booking→payment→tracking; vendor onboarding→accept order
UAT	Staging env + scripted scenarios	Real device/browser pass with pilot LaundryMart partners before launch

18. Deployment

- **Frontend:** Vercel, auto-deploy on merge to `main`, preview deployments per PR.
- **Backend:** Render or Railway, containerized Node service; health-check endpoint `/api/health`.
- **Database:** MongoDB Atlas, M10+ cluster with automated daily backups and point-in-time recovery.
- **Images:** Cloudinary, auto-format/quality (f_auto,q_auto) delivery.
- **Domain:** apex + www on Vercel; `api.` subdomain pointed at the backend host.
- **Env vars:** managed per-environment in each platform's secret store; never in source control.
- **CI/CD:** GitHub Actions — lint → unit/integration tests → Newman API tests → build → deploy; block merge on red pipeline.
- **Monitoring/Logging:** Sentry for frontend+backend error tracking; structured JSON logs (Winston/Pino) shipped to the hosting platform's log stream; Stripe Dashboard + webhook delivery logs monitored for failed events.

19. Non-Functional Requirements

- **Performance:** geo-search API p95 < 400ms; dashboard pages first-contentful-paint < 1.5s on 4G.

- **Availability:** 99.5% uptime target for API; graceful degradation (cached last-known vendor list) if geo-search is briefly unavailable.
 - **Scalability:** stateless API instances behind the host's load balancer; MongoDB indexes reviewed at 100k+ orders.
 - **Accessibility:** WCAG 2.1 AA — keyboard navigable, focus-visible states, form errors announced via `aria-live`.
 - **Responsiveness:** full functionality at 320px-2560px; dashboards use a collapsible sidebar < 1024px.
-

20. Assumptions, Risks & Dependencies

Assumptions

- Pickup/delivery logistics are handled by each LaundryMart itself (own staff/couriers) — the platform does not dispatch drivers in MVP.
- Single currency (USD) and single-region (US) launch; tax calculated as a flat configurable rate, not full nexus-based tax compliance.
- Vendors are individually vetted/onboarded manually before their Stripe Connect account is approved for payouts (Stripe's own risk review).

Risks

- No admin panel in MVP means disputes/fraud require direct database intervention — mitigate with a minimal internal read-only ops view before public launch if order volume is non-trivial.
- Vendor-managed logistics means delivery time reliability varies by vendor — surfaced to Clients via the vendor's rating/turnaround stats rather than a platform guarantee.
- Stripe Connect Express onboarding drop-off could bottleneck vendor supply — track completion rate and add reminder emails.

Dependencies

- Google Geocoding/Places API key (address entry + geocoding)
- Stripe account (Connect enabled) with live + test keys
- Cloudinary account for media storage
- Transactional email provider (e.g. SendGrid/Postmark) for auth + notification emails

21. Milestones & Development Roadmap

Phase	Scope	Est. Duration
1 — Foundations	Repo scaffolding, DB schemas, auth (both roles), RBAC middleware, CI pipeline	2 weeks
2 — Vendor Core	Vendor registration + Stripe Connect onboarding, business profile, services, availability	2 weeks
3 — Client Discovery	Geo search, vendor details, filters/sort	1.5 weeks
4 — Booking & Payments	Booking flow, checkout, Stripe PaymentIntent + webhooks, order lifecycle	2.5 weeks
5 — Order Ops	Manage Orders dashboard, status tracking, notifications, cancellations/refunds	2 weeks
6 — Trust & Polish	Reviews, earnings dashboard, profile/settings, public pages, accessibility pass	1.5 weeks
7 — Hardening & UAT	Full test suite, security review, pilot vendor UAT, staging soak	2 weeks
8 — Launch	Production deploy, monitoring live, phased city rollout	1 week

Total: ~14.5 weeks with one full-stack pair; parallelizable to ~9–10 weeks with a dedicated frontend + backend engineer working concurrently from Phase 2 onward.